

VINCIT DATA

Sistema Inteligente de Recomendación de Películas con Machine Learning, Deep Learning, MLOps y RAG

Integrantes:

Camila Nicole Rodas Miranda
Franco Jhoel Parra Aguilar
Alvaro Ariel Torrez Calle
Kael Alessandro Lopez Castillo
Esmeralda Paula Medina Paredes

Asignatura: Machine Learning
Docente: Paton Gutierrez, Ovidio Roger

Dataset: MovieLens 25M
Metodología: CRISP-DM + MLOps
Sistema: OmniRec-Movies / OmniCine

La Paz, Bolivia
Junio 2026

Índice

Resumen ejecutivo	1
1. Objetivo General	1
2. Objetivos Específicos	1
3. Descripción Preliminar de los Tres Mini-Proyectos	1
3.1. Mini Proyecto de Machine Learning (Clásico + AutoML)	1
3.2. Mini Proyecto de Deep Learning	2
3.3. Mini Proyecto de MLOps + RAG/Recuperación	2
4. Métricas Propuestas	2
4.1. Predicción de ratings	2
4.2. Recomendación Top-N	3
4.3. Recuperación semántica, RAG y MLOps	3
5. Posibles Riesgos Metodológicos y Mitigación	3
6. Estructura Propuesta de GitHub (para facilitar AutoML y MLOps)	3
7. Arquitectura General del Sistema	4
8. Business Understanding	5
8.1. Problema	5
8.2. Restricciones	6
8.3. Criterios de Éxito	6
9. Data Understanding	6
9.1. Procedencia del dataset	6
9.2. Descripción del Dataset	6
9.2.1. Dataset: Movies	6
9.2.2. Dataset: Ratings	7
9.2.3. Dataset: Tags	7
9.2.4. Dataset: Genome Scores y Genome Tags	7
9.2.5. Dataset: Links	7
9.3. Volumen de Datos	7
9.4. Cobertura	7
9.5. Tipos de Datos	7
9.6. Limitaciones	7
10. Análisis Exploratorio de Datos (EDA)	8
10.1. Visualizaciones e Insights	8
10.1.1. Distribución de Ratings	8
10.1.2. Actividad de usuarios	8
10.1.3. Popularidad de películas	9
10.1.4. Distribución de géneros	10
10.1.5. Evolución temporal	11
10.1.6. Tags libres y Tag Genome	11
10.1.7. Correlación entre métricas agregadas	12
10.2. Conclusiones del EDA	13

11.Data Preparation	13
11.1. Transformaciones y Limpieza de Variables	13
11.2. Partición del Dataset	13
11.3. Pipeline y Reproducibilidad	13
12.Modeling	14
12.1. Selección de Modelos	14
12.2. Configuración y Parámetros	14
12.3. Modelo Ganador y Justificación	14
12.4. Proceso de Entrenamiento	14
13.Evaluation	15
13.1. Justificación de Métricas	15
13.2. Protocolo de Comparación Justa	15
13.3. Resultados Cuantitativos y Análisis de Desempeño	15
13.4. AutoML como Benchmark Riguroso	16
13.5. Deep Learning	16
13.6. Análisis de Error y Segmentación	17
13.7. Clustering No Supervisado sobre Embeddings Latentes	18
13.8. Recuperación Semántica y RAG	20
13.8.1. Ejemplos concretos de consultas RAG	21
13.9. Límites y Generalización	22
13.10 Limitaciones éticas y sesgos	22
13.11 Trabajo futuro	22
14.Deployment	23
14.1. Arquitectura del Sistema	23
14.2. Flujos de Inferencia	24
14.2.1. 1. Motor de Recomendaciones	24
14.2.2. 2. Perfilado de Comunidad	24
14.2.3. 3. Similitud de Películas	24
14.2.4. 4. Búsqueda Semántica y RAG	24
14.3. Ciclo de Vida del Usuario	24
14.4. MLOps: Pipeline, Tracking y Registry	24
14.5. Monitoreo y Política de Reentrenamiento	25
14.6. Especificaciones Técnicas de Despliegue	25
15.Conclusiones	25

Resumen ejecutivo

OmniRec-Movies es un sistema de recomendación de películas desarrollado sobre MovieLens 25M siguiendo la metodología CRISP-DM de extremo a extremo. El informe anterior documentaba principalmente las fases de Machine Learning clásico y Deep Learning; esta versión final integra además la fase completa de MLOps, recuperación semántica y RAG, junto con el despliegue como plataforma web. La solución final combina modelos clásicos de filtrado colaborativo, modelos neuronales de embeddings, clustering de comunidades, búsqueda semántica con transformers e inferencia servida mediante FastAPI, Next.js, MLflow, Docker y un registro de artefactos versionado.

1. Objetivo General

Desarrollar un sistema completo de recomendación de películas que integre modelos clásicos de Machine Learning, modelos de Deep Learning, un flujo de AutoML como benchmark obligatorio, una capa de MLOps reproducible y un módulo de recuperación semántica con RAG. El sistema debe ser capaz de recomendar películas personalizadas, resolver escenarios de usuario nuevo, permitir búsqueda en lenguaje natural y demostrar trazabilidad técnica mediante tracking de experimentos, versionado de artefactos, pipeline de inferencia y monitoreo.

2. Objetivos Específicos

1. Predecir ratings o rankings personalizados comparando enfoques clásicos de recomendación contra un baseline explícito.
2. Ejecutar AutoML como benchmark metodológico para comparar el rendimiento y costo del modelado manual contra la selección automática.
3. Implementar modelos profundos con embeddings usuario-ítem para capturar relaciones latentes y mejorar la similitud entre películas.
4. Construir un buscador semántico basado en descriptores de películas, géneros y Tag Genome, con recuperación vectorial y respuesta generativa opcional.
5. Establecer un flujo MLOps reproducible con configuración como código, tracking de experimentos, registro de modelos, hashes de integridad y monitoreo.
6. Desplegar una plataforma web funcional que traduzca cada modelo en una experiencia concreta para el usuario final.

3. Descripción Preliminar de los Tres Mini-Proyectos

La estructura original del proyecto se mantiene: tres mini-proyectos técnicos y una integración final. La diferencia principal respecto al informe anterior es que ahora las fases ya no están solamente planificadas; se documentan como componentes implementados y evaluados.

3.1. Mini Proyecto de Machine Learning (Clásico + AutoML)

- **EDA y preparación de datos:** se trabajó con MovieLens 25M usando procesamiento columnar, archivos Parquet y muestreo estratificado para preservar la distribución de actividad de usuarios.
- **Baseline explícito:** recomendador por popularidad bayesiana ponderada, usado como piso de comparación y como fallback para usuarios sin historial.
- **Modelos clásicos:** KNN-Baseline item-based, SVD y NMF usando *scikit-surprise*.

- **AutoML:** búsqueda automática sobre familias de modelos e hiperparámetros. Su rol fue validar el proceso de selección, no reemplazar el análisis manual.
- **Componente no supervisado:** KMeans sobre factores latentes para obtener comunidades de usuarios y grupos de películas.
- **Resultado:** SVD quedó como modelo clásico principal por su balance entre RMSE, latencia, interpretabilidad y capacidad de *folding-in* para usuarios nuevos.

3.2. Mini Proyecto de Deep Learning

- **Modelo profundo principal:** MF+Bias en PyTorch, equivalente neuronal de una factorización matricial con embeddings de usuarios e ítems, sesgos y salida acotada al rango de ratings.
- **Modelo comparativo:** NeuMF, arquitectura Neural Collaborative Filtering con rama GMF y rama MLP.
- **Embeddings:** se usaron representaciones latentes de películas para similitud semántica no supervisada.
- **Resultado:** en comparación justa sobre el mismo *split*, MF+Bias alcanzó RMSE 0,8046 y MAE 0,6113, superando al SVD en ese protocolo.
- **Uso final:** el modelo profundo se utiliza principalmente para la funcionalidad de películas similares, donde no sufre el mismo problema de *cold-start* de usuario.

3.3. Mini Proyecto de MLOps + RAG/Recuperación

- **Estructura reproducible:** repositorio organizado en notebooks, código productivo, configuración YAML, artefactos, modelos, API, Docker y frontend.
- **MLOps:** pipeline idempotente con etapas de preparación, entrenamiento, entrenamiento profundo, baseline, indexación, evaluación, registro y monitoreo.
- **Tracking:** MLflow registra métricas, parámetros y runs de entrenamiento/evaluación. El experimento final documentado es `omnirec-fase4-mlops-rag`.
- **Versionado:** los artefactos de servicio se guardan con manifiestos SHA256, versión actual y linaje.
- **RAG y recuperación:** se implementó un encoder Sentence-Transformers MiniLM, un índice vectorial FAISS y una capa generativa opcional con LLM. La propuesta inicial mencionaba ChromaDB; la implementación final usa FAISS como índice local de alta eficiencia, manteniendo el objetivo de recuperación por similitud semántica.
- **Demo funcional:** backend FastAPI, frontend Next.js 16 y orquestación con Docker Compose.

4. Métricas Propuestas

Las métricas propuestas inicialmente se mantienen, pero ahora se reportan junto con el protocolo usado en cada fase.

4.1. Predicción de ratings

- **RMSE:** métrica principal porque penaliza con mayor fuerza los errores grandes.
- **MAE:** error promedio absoluto en escala de estrellas, más interpretable para usuarios finales.

4.2. Recomendación Top-N

- **Precision@5 y Precision@10:** proporción de recomendaciones relevantes dentro del top entregado.
- **Recall@5 y Recall@10:** cobertura de ítems relevantes recuperados.
- **NDCG@10:** calidad del ranking considerando posición y relevancia.

4.3. Recuperación semántica, RAG y MLOps

- **Precision@k semántico:** proporción de resultados recuperados que son coherentes con la consulta.
- **Cobertura del Tag Genome:** porcentaje de películas con descriptores suficientes para recuperación.
- **Latencia y costo de entrenamiento:** tiempo de entrenamiento y costo operativo de cada modelo.
- **Drift y degradación:** comparación contra la versión `current` del registry usando umbrales de alerta.

5. Posibles Riesgos Metodológicos y Mitigación

Riesgo	Probabilidad	Impacto	Mitigación
Memoria insuficiente con 25 millones de ratings	Alta	Alto	Uso de Parquet, procesamiento columnar, muestreo estratificado y datasets operativos reducidos.
AutoML demasiado lento en dataset completo	Alta	Medio	Ejecutar AutoML sobre muestra representativa y usarlo como benchmark, no como flujo productivo.
Sobreajuste en Deep Learning	Medio	Alto	Early stopping, validación separada, salida acotada y comparación contra SVD en el mismo split.
Cold-start de usuarios o películas	Alta	Alto	Baseline bayesiano para usuarios nuevos, SVD con folding-in, RAG para búsqueda semántica y fallback de similitud.
Sesgo de popularidad	Alta	Medio	Evaluación por popularidad, análisis de long tail y combinación con descriptores semánticos.
Dificultad de reproducir el sistema	Medio	Alto	Docker, requirements fijados, configuración YAML, semilla 42 y pipeline MLOps.
Pérdida de trazabilidad de modelos	Medio	Alto	MLflow, registry local, manifiestos SHA256, alias <code>current</code> y linaje de artefactos.
Drift de datos en producción	Medio	Medio	Monitoreo PSI, alertas de frescura, cobertura y degradación de RMSE/NDCG.

6. Estructura Propuesta de GitHub (para facilitar AutoML y MLOps)

La estructura final conserva la intención presentada al inicio, pero añade los componentes productivos que no estaban completos en el informe anterior.

```

movie-recommendation-crispdm/
├─ notebooks/      # 6 notebooks CRISP-DM + AutoML
├─ src/            # código limpio (modelos, rag, utils)
├─ configs/        # hiperparámetros YAML
├─ models/         # modelos guardados (.pkl, .pt)
├─ artifacts/      # datasets Parquet, embeddings
├─ mlruns/         # tracking MLflow
├─ vector_store/   # ChromaDB para RAG
├─ api/            # FastAPI
├─ docker/         # Dockerfile + compose
├─ requirements.txt
├─ Dockerfile
└─ README.md       # instrucciones paso a paso + cómo correr demo

```

Figura 1: Estructura propuesta del repositorio para notebooks, código productivo, artefactos, vector store, API y despliegue.

```

movie-recommendation-crispdm/
|-- notebooks/      # CRISP-DM, AutoML, Deep Learning, RAG
|-- src/            # pipeline, modelos, registry, evaluacion, monitoreo
|-- config/         # hiperparametros YAML y umbrales MLOps
|-- models/         # modelos .pkl, .pt y registry versionado
|-- artifacts/      # datasets Parquet, embeddings y reportes
|-- mlruns/         # tracking MLflow
|-- vector_store/   # indice semantico / artefactos RAG
|-- app/backend/    # FastAPI
|-- app/web-app/    # Next.js / OmniCine
|-- docker/         # Dockerfile, compose, entrypoint
|-- requirements.txt
|-- README.md

```

7. Arquitectura General del Sistema

El sistema se organiza como un flujo de datos, entrenamiento, registro, servicio y monitoreo. La arquitectura separa el entrenamiento offline de la inferencia online, permitiendo que los modelos se evalúen, versionen y sirvan de forma reproducible.

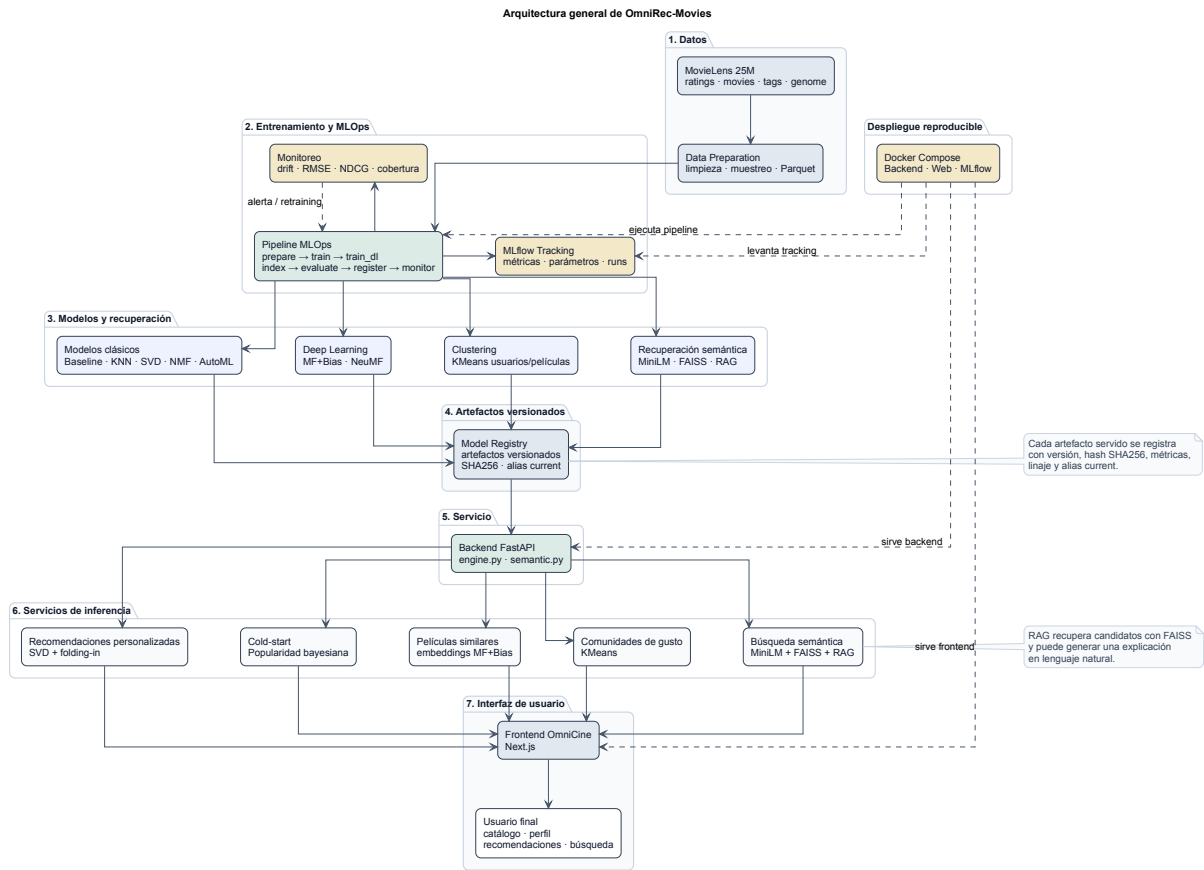


Figura 2: Arquitectura general de OmniRec: datos, pipeline MLOps, modelos, registry, backend, frontend, monitoreo y despliegue.

8. Business Understanding

8.1. Problema

El reto consiste en construir un sistema de recomendación de películas que no dependa de un único enfoque. Una plataforma real de entretenimiento debe resolver varios escenarios: usuarios sin historial, usuarios con pocas calificaciones, usuarios activos, búsqueda por intención semántica y necesidad de explicar o diversificar recomendaciones. Por ello, OmniRec combina popularidad bayesiana, factorización matricial, embeddings profundos, clustering y búsqueda semántica.

El problema se vuelve más complejo por tres obstáculos técnicos propios de los sistemas de recomendación. Primero, la **dispersión**: aunque MovieLens 25M contiene millones de calificaciones, la mayoría de combinaciones posibles usuario–película no tiene interacción registrada. Segundo, el **cold-start**: un usuario nuevo o una película poco evaluada no ofrece suficiente evidencia colaborativa para personalizar con confianza. Tercero, el **sesgo de popularidad**: si el sistema solo sigue la señal histórica dominante, tiende a repetir películas masivas y reduce la visibilidad de contenido de nicho. Estos tres problemas justifican el diseño híbrido del proyecto: baseline bayesiano para casos sin historial, SVD y Deep Learning para preferencias latentes, y RAG para búsqueda por significado.

Desde el punto de vista de negocio, un recomendador mejora la retención y satisfacción del usuario porque reduce el esfuerzo de exploración del catálogo. En un entorno competitivo de entretenimiento, recomendar contenido relevante ayuda a aumentar interacción, permanencia y percepción de valor. La variable de preferencia principal es el rating explícito, usado para estimar afinidad y producir rankings personalizados.

8.2. Restricciones

- Se utiliza el dataset MovieLens 25M desde la fuente oficial de GroupLens.
- La solución debe poder ejecutarse con recursos razonables, sin depender de hardware especializado de alto costo.
- La metodología CRISP-DM estructura el informe y el ciclo de trabajo.
- Los transformers se usan en modo inferencia, no como entrenamiento masivo desde cero.
- El sistema debe justificar explícitamente cómo maneja usuarios o ítems poco frecuentes.
- Todo artefacto servido debe ser reproducible, versionable y monitoreable.

8.3. Criterios de Éxito

1. Los modelos desarrollados deben superar un baseline explícito de popularidad o similitud.
2. AutoML debe ejecutarse como benchmark sobre una partición comparable.
3. Las métricas deben ser coherentes con el dominio: RMSE/MAE para ratings, Precision/Recall/NDCG para ranking y métricas de cobertura para recuperación.
4. El sistema debe documentar sus errores, límites y escenarios de cold-start.
5. El proyecto debe mostrar continuidad entre Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation y Deployment.
6. Un tercero debe poder ejecutar entrenamiento, inferencia y demo siguiendo las instrucciones del repositorio.
7. MLflow, registry, hashes y monitoreo deben cubrir los artefactos usados en producción.
8. Deben existir ejemplos y mecanismos de búsqueda semántica/RAG para consultas en lenguaje natural.

9. Data Understanding

9.1. Procedencia del dataset

El conjunto de datos utilizado es MovieLens 25M, publicado por GroupLens Research. Contiene 25.000.095 calificaciones y 1.093.360 aplicaciones de etiquetas realizadas por 162.541 usuarios sobre 62.423 películas. Los datos fueron recolectados entre el 9 de enero de 1995 y el 21 de noviembre de 2019, y la versión utilizada fue publicada en diciembre de 2019 [Harper and Konstan, 2015].

El dataset fue descargado desde la fuente oficial de GroupLens: <https://grouplens.org/datasets/movielens/25m/>. Para reproducibilidad, se conserva la referencia al ZIP original <https://files.grouplens.org/datasets/movielens/ml-25m.zip>.

9.2. Descripción del Dataset

A diferencia del informe anterior, esta versión final no se limita a `movies.csv` y `ratings.csv`. La fase de RAG y recuperación semántica requiere integrar los seis archivos principales del dataset.

9.2.1. Dataset: Movies

- `movieId`: identificador único de película.
- `title`: título con año de lanzamiento.
- `genres`: lista de géneros separada por delimitadores.

9.2.2. Dataset: Ratings

- `userId`: identificador del usuario.
- `movieId`: película evaluada.
- `rating`: calificación explícita entre 0,5 y 5,0.
- `timestamp`: momento de la interacción.

9.2.3. Dataset: Tags

Contiene etiquetas libres escritas por usuarios. Es útil para comprender lenguaje natural, preferencias explícitas y vocabulario usado por la comunidad.

9.2.4. Dataset: Genome Scores y Genome Tags

`genome-scores.csv` relaciona películas con tags del genoma mediante una relevancia numérica. `genome-tags.csv` contiene el diccionario de 1.128 tags. Estos archivos son fundamentales para construir descriptores semánticos por película.

9.2.5. Dataset: Links

Relaciona `movieId` con identificadores externos como IMDb y TMDb. En el proyecto final no fue el centro del modelado, pero permite extender la solución con metadatos externos.

9.3. Volumen de Datos

El volumen del dataset exige decisiones de ingeniería: procesamiento columnar, muestreo estratificado, compresión Parquet y separación entre datasets de exploración, entrenamiento y servicio. En producción se utiliza una muestra operativa manejable, mientras que el análisis conserva trazabilidad hacia MovieLens 25M.

9.4. Cobertura

La cobertura se concentra en el dominio cinematográfico. Los géneros dan una caracterización básica, mientras que tags libres y Tag Genome aportan mayor granularidad semántica. Esta combinación permite pasar de un recomendador puramente colaborativo a un sistema híbrido con recuperación por significado.

9.5. Tipos de Datos

- Numéricos discretos: `userId`, `movieId`, `tagId`.
- Numéricos continuos: `rating`, `relevance`.
- Categóricos y multietiqueta: `genres`.
- Temporales: `timestamp`.
- Textuales: `title`, `tag`.

9.6. Limitaciones

- La matriz usuario–película es altamente dispersa.
- No existen atributos demográficos de usuarios.
- Los ratings son feedback explícito; no hay clicks, reproducciones ni tiempo de visualización.
- Existe sesgo de popularidad hacia películas conocidas.

- El Tag Genome está en inglés, por lo que la búsqueda multilingüe depende de la capacidad del encoder.
- El dataset es estático y no refleja cambios recientes del mercado cinematográfico.

10. Análisis Exploratorio de Datos (EDA)

El EDA permitió entender la distribución de ratings, la actividad de usuarios, el sesgo de popularidad, la composición de géneros, la evolución temporal y la riqueza semántica de los tags.

10.1. Visualizaciones e Insights

10.1.1. Distribución de Ratings

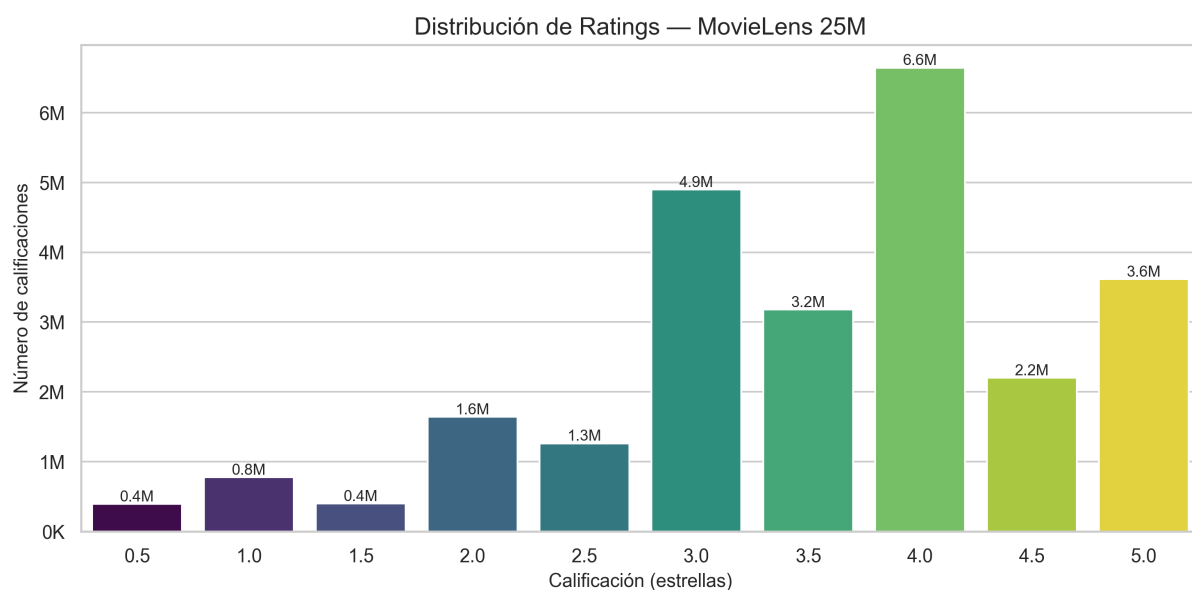


Figura 3: Distribución de ratings en MovieLens 25M.

Las calificaciones se concentran en valores altos, especialmente entre 3,0 y 5,0. Esto confirma un sesgo positivo típico: los usuarios tienden a calificar contenido que conocen o que les generó suficiente interés.

10.1.2. Actividad de usuarios

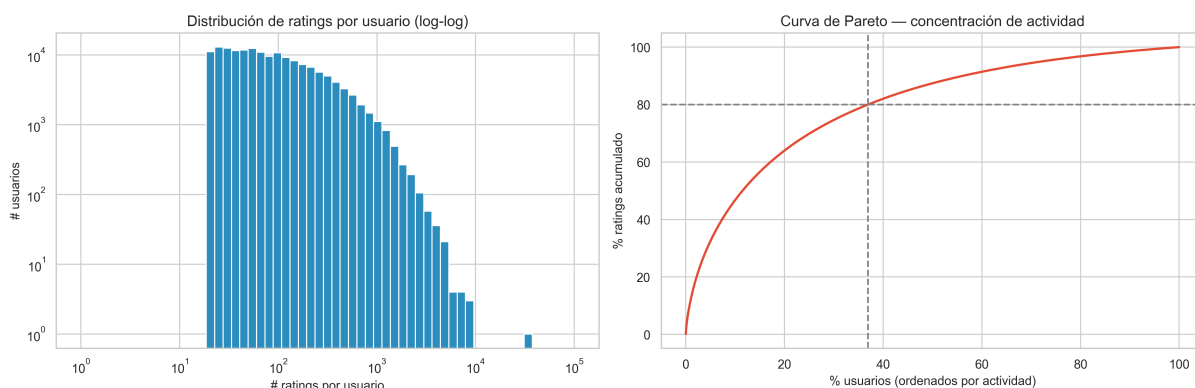


Figura 4: Distribución de ratings por usuario y concentración de actividad.

La actividad sigue una distribución de cola larga: pocos usuarios concentran una parte importante de las calificaciones. Este patrón justifica el muestreo estratificado por niveles de actividad.

10.1.3. Popularidad de películas

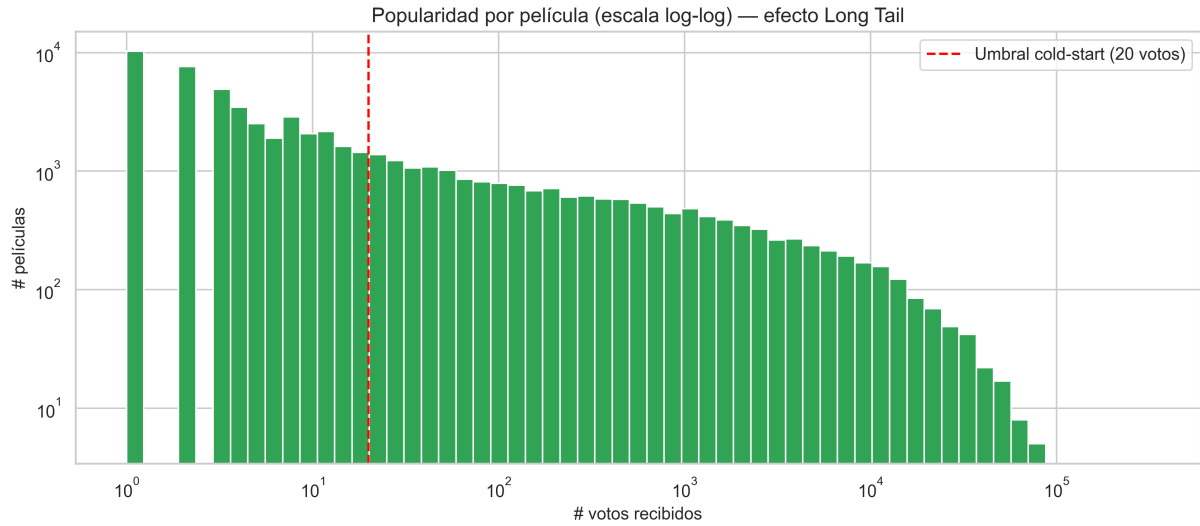


Figura 5: Popularidad por película y efecto long tail.

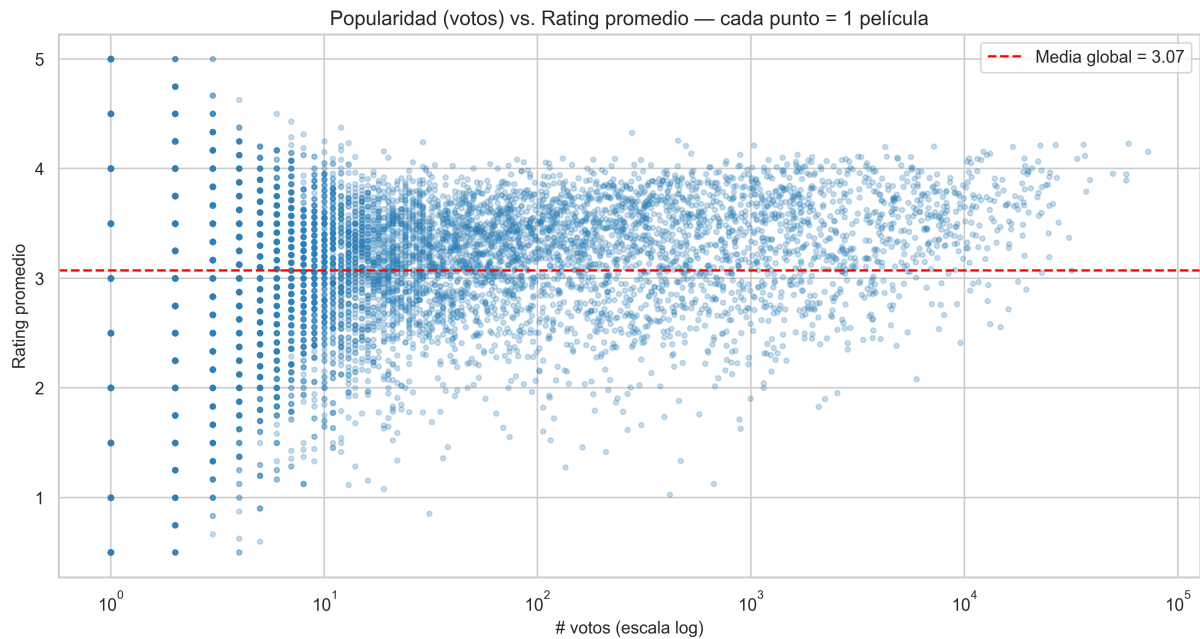


Figura 6: Relación entre número de votos y rating promedio.

La mayoría de las películas tiene pocas interacciones, mientras que un grupo reducido concentra miles de votos. Esta condición afecta a KNN, SVD y NMF, y explica la necesidad de estrategias para cold-start y long tail.

10.1.4. Distribución de géneros

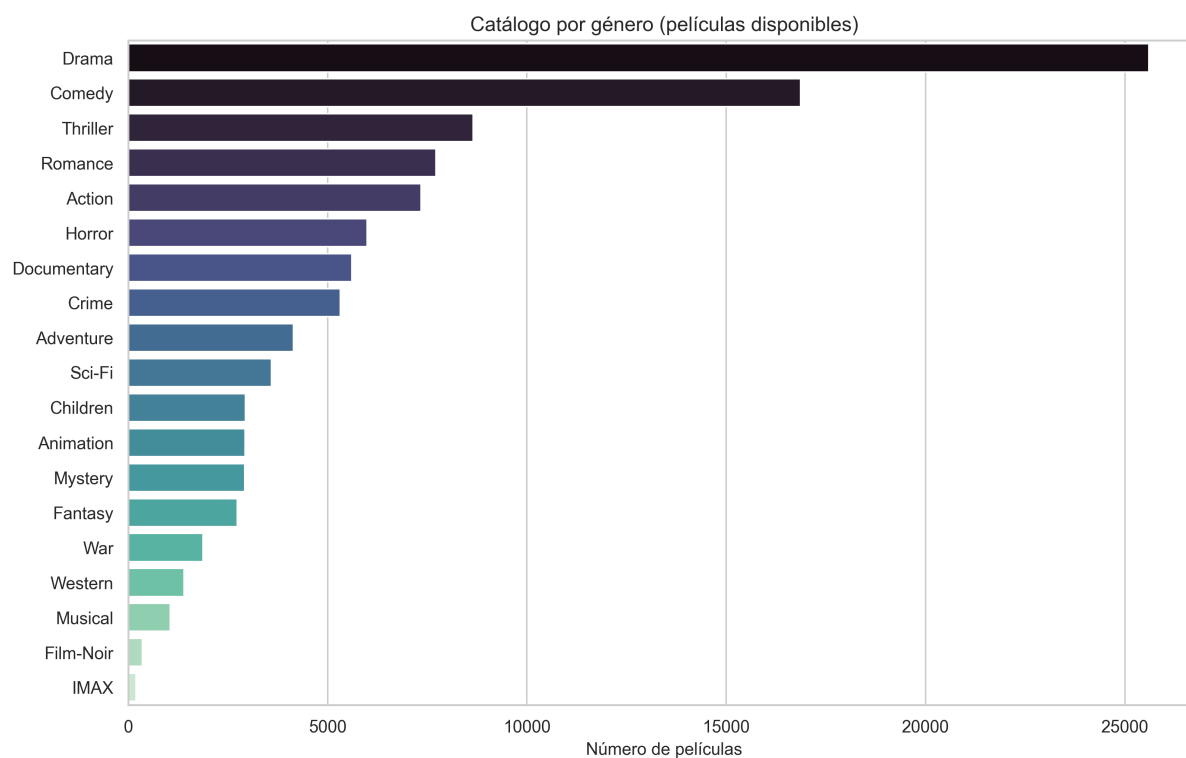


Figura 7: Catálogo por género.

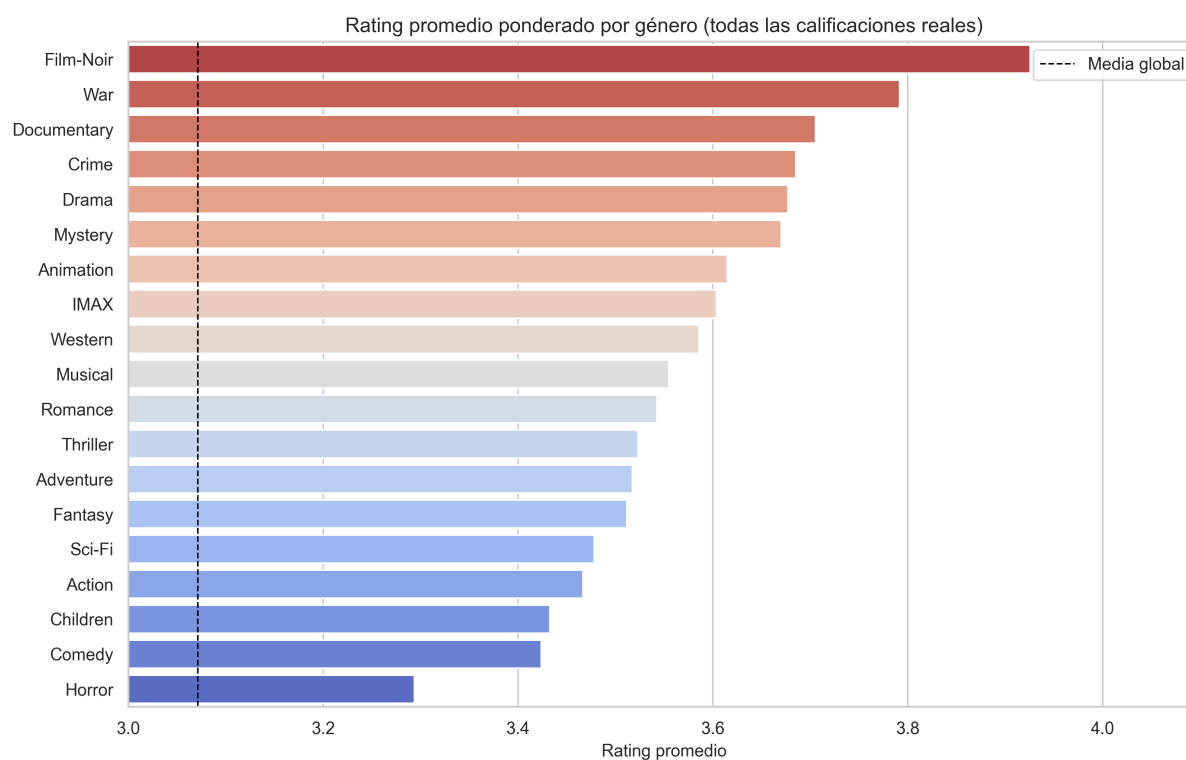


Figura 8: Rating promedio ponderado por género.

Drama y Comedy dominan el catálogo, mientras que géneros como Film-Noir, War y Documentary muestran ratings ponderados altos. Esta diferencia ayuda a interpretar sesgos de catálogo y preferencias agregadas.

10.1.5. Evolución temporal

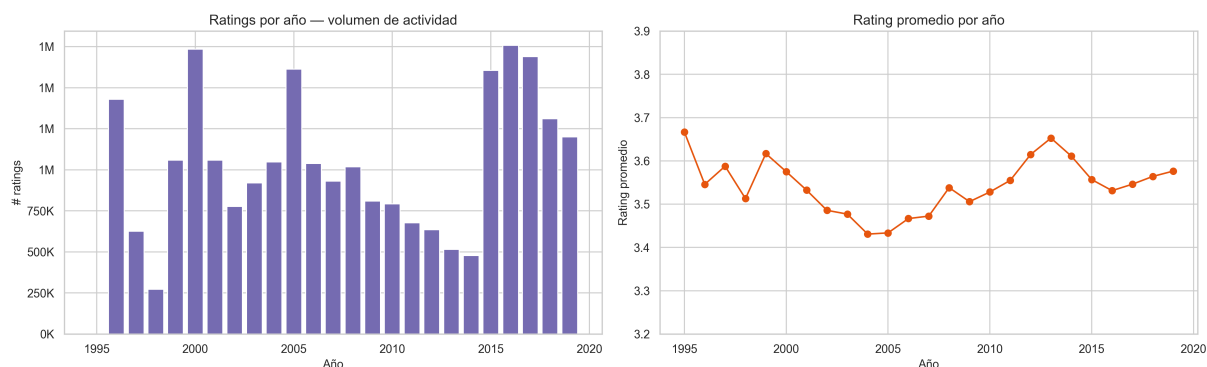


Figura 9: Volumen de ratings y rating promedio por año.

El volumen anual de ratings varía fuertemente, lo que confirma que el tiempo debe considerarse al evaluar. Por eso se usó split temporal en lugar de un split aleatorio tradicional.

10.1.6. Tags libres y Tag Genome

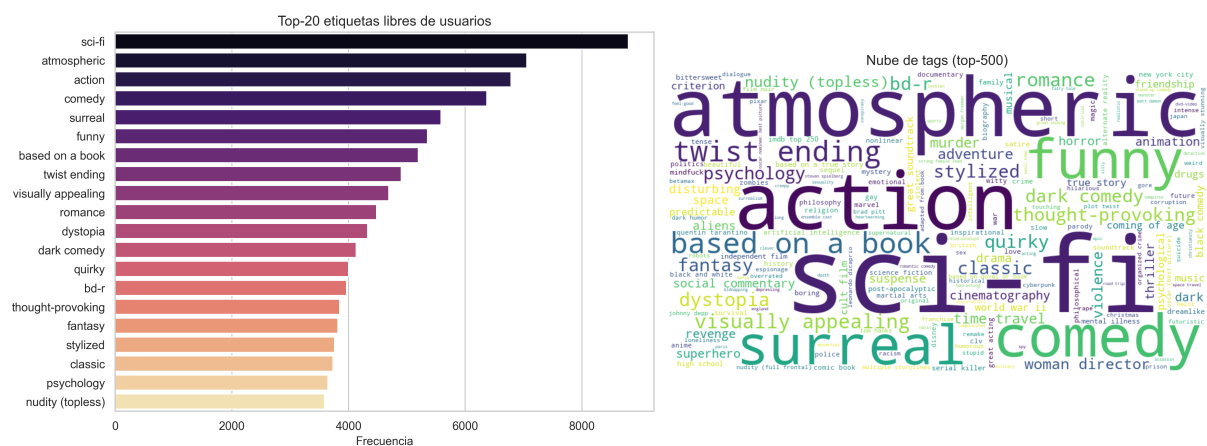


Figura 10: Top-20 etiquetas libres de usuarios y nube de tags.

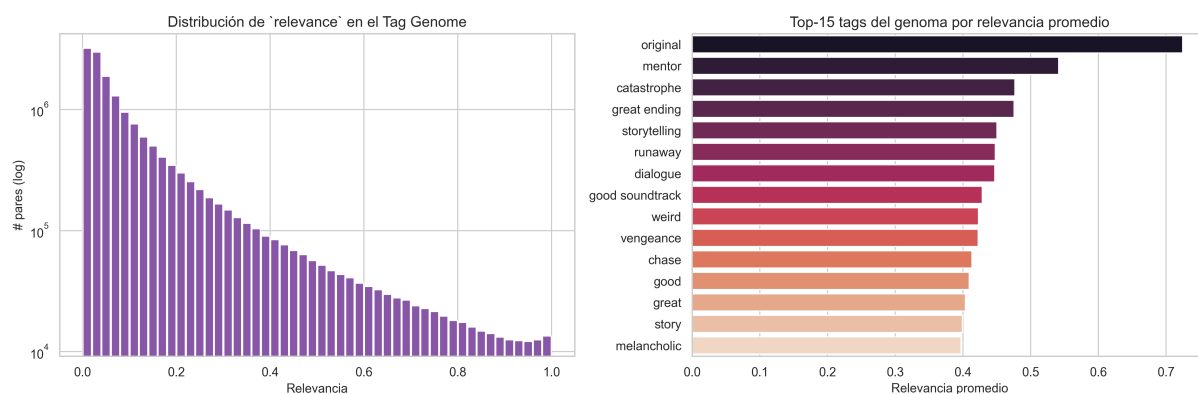


Figura 11: Distribución de relevancia en Tag Genome y tags con mayor relevancia promedio.

Los tags aportan una señal semántica que no aparece en los ratings. Esta señal permite construir descriptores textuales por película y habilitar búsqueda en lenguaje natural.

10.1.7. Correlación entre métricas agregadas

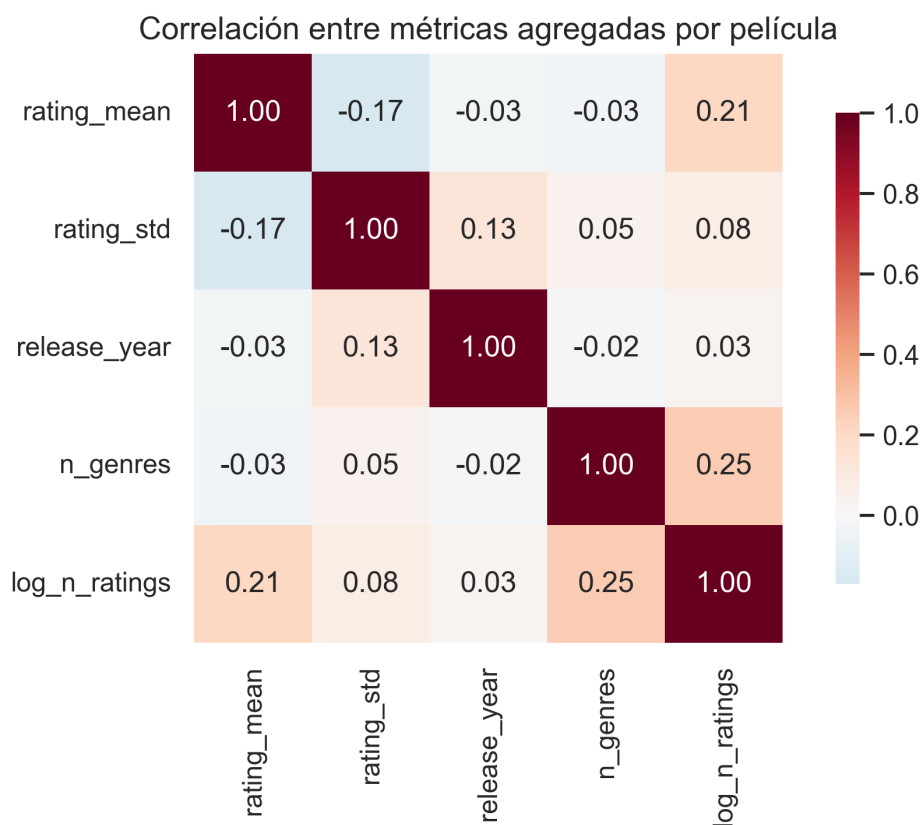


Figura 12: Correlación entre métricas agregadas por película.

La correlación entre rating promedio, número de géneros, año y popularidad es limitada. Esto sugiere que no basta con reglas simples sobre metadatos; se necesitan modelos colaborativos y representaciones latentes.

10.2. Conclusiones del EDA

El EDA confirma cuatro decisiones técnicas: usar factorización matricial para manejar dispersión, incluir un baseline robusto para usuarios nuevos, aplicar muestreo estratificado para conservar la estructura de actividad y construir una capa semántica con tags para cubrir consultas que no pueden resolverse solamente con ratings.

11. Data Preparation

Esta fase transforma los datos originales en artefactos reproducibles para modelado, recuperación e inferencia.

11.1. Transformaciones y Limpieza de Variables

- **Categorización de usuarios:** se crearon tiers de actividad para muestreo estratificado.
- **Manejo de nulos:** se filtraron valores faltantes marginales en tags e identificadores externos.
- **Filtro cold-start:** se descartaron películas con muy pocos votos en ciertos experimentos para reducir ruido.
- **Normalización para baseline:** se aplicó popularidad bayesiana ponderada con shrinkage hacia la media global.
- **Factores latentes:** los embeddings de SVD se usaron para clustering y perfilado de comunidad.
- **Descriptores semánticos:** para RAG se construyó un texto por película combinando título, géneros y top tags del genoma.

11.2. Partición del Dataset

- **Muestra principal 60 %:** usada por SVD, NMF, AutoML y evaluación clásica.
- **Muestra 10 %:** reservada para KNN por su alto costo computacional.
- **Split temporal leave-last-1-out:** el último rating de cada usuario queda como test, simulando evaluación cronológica.
- **Split Deep Learning 70/15/15:** usado para comparar MF+Bias, NeuMF y SVD clásico bajo el mismo protocolo.

11.3. Pipeline y Reproducibilidad

El pipeline usa semilla global 42, rutas configurables en YAML, Parquet para persistencia, Docker para paridad de entorno y MLflow para registrar ejecuciones. Los principales artefactos preparados son:

Artefacto	Tamaño	Uso
ratings_knn_10pct.parquet	2,47 M filas	Dataset operativo de servicio y entrenamiento manejable.
movies_prepared_60pct.parquet	55.113 filas	Catálogo preparado para inferencia.
genome_scores_prepared_60pct.parquet	15,58 M filas	Relevancias tag-película para RAG.
genome_tags.parquet	1.128 filas	Diccionario del Tag Genome.
item_clusters.parquet / user_clusters.parquet	–	Clústeres y estadísticas de comunidad.

Cuadro 2: Artefactos derivados de preparación de datos.

12. Modeling

La fase de modelado evolucionó desde una comparación clásica hacia un sistema híbrido. Cada modelo se conserva si cumple un rol claro: benchmark, recomendación personalizada, similitud, perfilado, búsqueda o fallback.

12.1. Selección de Modelos

Modelo	Estado	Rol en el sistema
Popularidad bayesiana	Producción	Baseline, ranking para invitados y fallback cold-start.
KNN-Baseline	Documentación	Comparador clásico por vecindad item-based.
SVD	Producción	Recomendación personalizada, afinidad y folding-in.
NMF	Documentación	Comparador de factorización no negativa.
AutoML	Benchmark	Validación de búsqueda de modelos e hiperparámetros.
KMeans	Producción	Comunidades de usuarios y grupos de películas.
MF+Bias	Producción	Similitud de películas mediante embeddings profundos.
NeuMF	Documentación	Comparador neuronal más complejo.
MiniLM + FAISS	Producción	Recuperación semántica en lenguaje natural.
RAG generativo	Producción opcional	Respuesta redactada sobre resultados recuperados.

Cuadro 3: Catálogo final de modelos y componentes.

12.2. Configuración y Parámetros

- **SVD:** 50 factores, 20 épocas, learning rate 0,005 y regularización 0,02.
- **KNN:** enfoque item-based con *pearson baseline*, $k = 40$ y soporte mínimo.
- **NMF:** factorización no negativa con regularización y número de factores menor al SVD.
- **MF+Bias:** embeddings de dimensión 32, optimizador Adam, salida acotada y early stopping.
- **NeuMF:** ramas GMF y MLP con capas densas, dropout y salida acotada.
- **RAG:** encoder `paraphrase-multilingual-MiniLM-L12-v2`, descriptores por película, vectores normalizados e índice FAISS con similitud coseno.

12.3. Modelo Ganador y Justificación

No existe un único modelo ganador universal; el sistema final asigna cada modelo al caso donde es más útil.

- **SVD** es el ganador clásico y el motor principal de recomendación personalizada por su eficiencia, RMSE bajo y capacidad de folding-in.
- **MF+Bias** es el mejor modelo en el protocolo de Deep Learning, con RMSE 0,8046, y se usa para similitud de películas.
- **Popularidad bayesiana** es el fallback correcto para usuarios sin historial.
- **MiniLM + FAISS + RAG** resuelve consultas semánticas que los modelos colaborativos no pueden responder directamente.

12.4. Proceso de Entrenamiento

El proceso final se ejecuta desde notebooks o desde el pipeline MLOps:

```
python -m src.pipeline all
python -m src.pipeline train_dl
python -m src.pipeline index
python -m src.pipeline evaluate
python -m src.pipeline register
```

El pipeline completo encadena preparación, entrenamiento clásico, entrenamiento profundo, baseline, indexación semántica, evaluación, registro y monitoreo. Los artefactos generados se guardan en `models/` y se versionan en `models/registry/`.

13. Evaluation

Esta sección resume los resultados cuantitativos y el análisis de desempeño. La interpretación se separa por protocolo para evitar comparaciones incorrectas entre fases.

13.1. Justificación de Métricas

RMSE y MAE permiten evaluar predicción de ratings. Precision, Recall y NDCG se incorporan porque el objetivo práctico es producir rankings Top-N. Para RAG se agregan métricas de recuperación y cobertura. Para MLOps se consideran drift, frescura, degradación y reproducibilidad.

13.2. Protocolo de Comparación Justa

1. Los modelos clásicos principales usan la misma muestra estratificada del 60 %.
2. KNN se evalúa sobre 10 % por costo computacional.
3. El split temporal leave-last-1-out evita fuga de información temporal.
4. Deep Learning se compara contra SVD usando el mismo split 70/15/15.
5. Los resultados entre Fase 3 y Fase 4 no se comparan directamente si el protocolo cambia.

13.3. Resultados Cuantitativos y Análisis de Desempeño

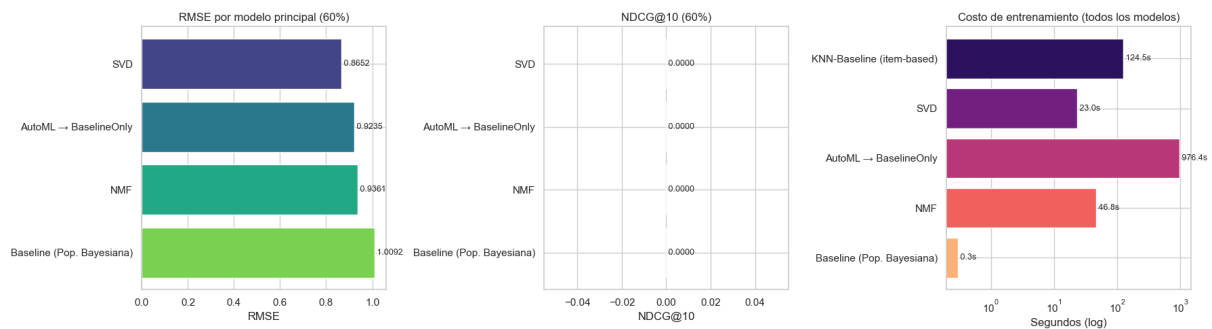


Figura 13: Comparación de RMSE, NDCG@10 y costo de entrenamiento en los modelos clásicos.

Modelo	Dataset	RMSE	MAE	P@10	R@10	Tiempo (s)
KNN-Baseline item-based	KNN 10 %	0,8935	0,6690	0,3012	0,3012	124,5
SVD	MAIN 60 %	0,8652	0,6561	0,2803	0,2803	23,0
AutoML → BaselineOnly	MAIN 60 %	0,9235	0,7044	0,2460	0,2460	976,4
NMF	MAIN 60 %	0,9361	0,7129	0,2482	0,2482	46,8
Baseline bayesiano	MAIN 60 %	1,0092	0,7924	0,1236	0,1236	0,3

Cuadro 4: Resultados de modelos clásicos a partir del CSV de métricas adjunto. En el archivo, NDCG@10 figura como 0,0000 para esta corrida; por ello la lectura principal de esta tabla se apoya en RMSE, MAE, Precision/Recall y costo.

SVD supera al baseline bayesiano, NMF y AutoML en RMSE y MAE bajo el protocolo clásico. KNN tiene métricas competitivas, pero fue entrenado sobre una muestra menor y tiene mayor costo de memoria/latencia, por lo que no es el candidato más adecuado para producción.

13.4. AutoML como Benchmark Riguroso

AutoML no se usó como sustituto del análisis técnico, sino como control metodológico. Su resultado confirmó que los modelos manuales, especialmente SVD, estaban cerca de una solución competitiva. El costo de entrenamiento de AutoML fue mucho mayor que el de SVD, lo que refuerza la decisión de usarlo como benchmark y no como componente servido.

Una conclusión metodológica importante es que la automatización no elimina la necesidad de criterio técnico. AutoML explora configuraciones y familias de modelos, pero su rendimiento depende del presupuesto de cómputo, del tamaño de muestra y del espacio de búsqueda definido. En este proyecto fue útil como evidencia de comparación: mostró que la configuración manual de SVD era competitiva, pero también que una búsqueda automática costosa puede quedar por debajo de un modelo más simple cuando los recursos son limitados. Por eso se mantuvo como benchmark científico y no como motor operativo del sistema.

13.5. Deep Learning

Modelo	Familia	Test RMSE	Test MAE
MF+Bias	Deep Learning	0,8046	0,6113
SVD en el mismo split	Factorización clásica	0,8194	0,6212
NeuMF	Deep Learning	0,8196	0,6208

Cuadro 5: Comparación de Deep Learning contra clásico en el mismo split.

La conclusión técnica cambia respecto al informe anterior: Deep Learning sí aporta valor cuando se evalúa bajo el mismo protocolo. MF+Bias supera al SVD en RMSE/MAE y sus embeddings de ítems son útiles para similitud de películas.

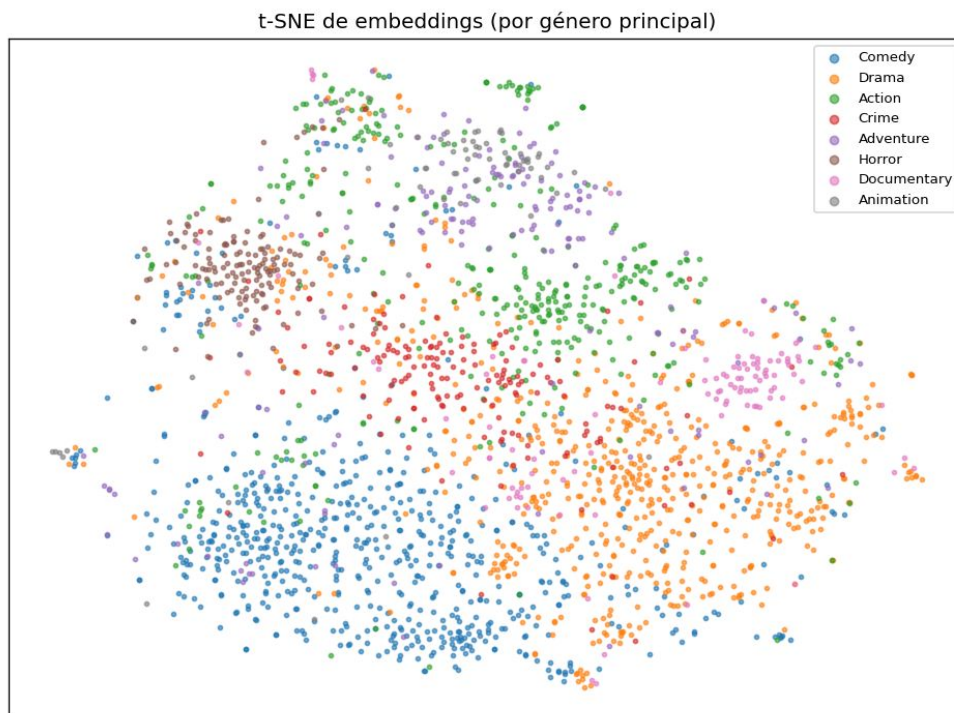


Figura 14: Proyección t-SNE de embeddings de películas coloreados por género principal.

La proyección t-SNE muestra que los embeddings aprendidos no son vectores arbitrarios: tienden a organizar películas cercanas por afinidad de contenido y género principal. Aunque existe solapamiento natural entre géneros, la presencia de zonas dominadas por Comedy, Drama, Action, Adventure y otros grupos confirma que el espacio latente captura estructura semántica útil para la función de películas similares.

13.6. Análisis de Error y Segmentación

- **Warm-start vs. cold-start:** el error baja cuando el usuario y la película ya tienen historial suficiente. Los casos cold-start justifican baseline y RAG.
- **Tiers de usuarios:** los usuarios con muchas interacciones permiten mejores perfiles; los usuarios casuales requieren priors y folding-in.
- **Head vs. long tail:** los modelos colaborativos favorecen películas populares y tienen mayor incertidumbre en la cola larga.

13.7. Clustering No Supervisado sobre Embeddings Latentes



Figura 15: Silhouette para clústeres de películas.

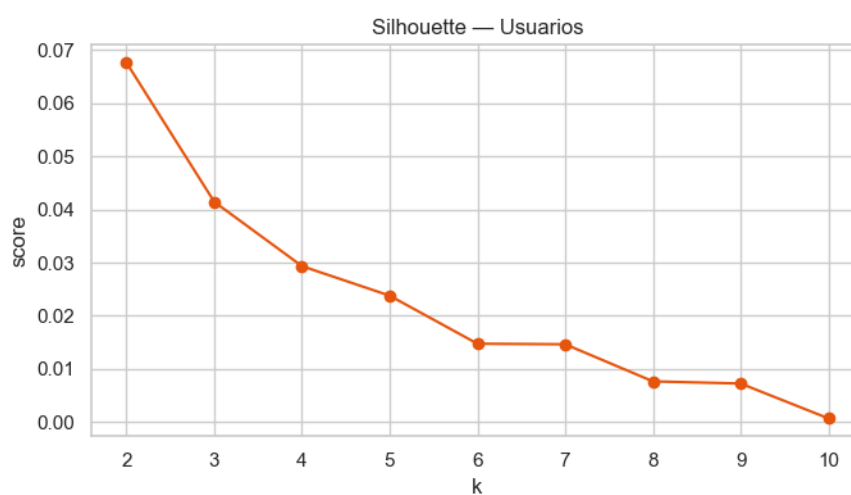


Figura 16: Silhouette para clústeres de usuarios.

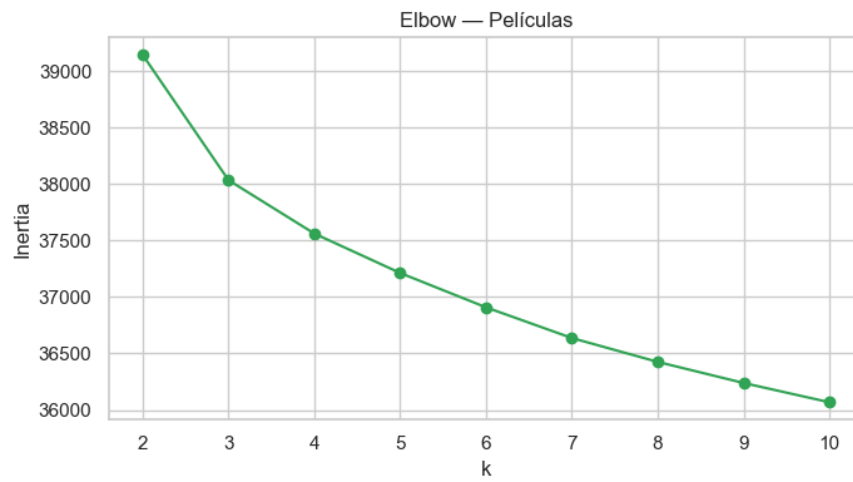


Figura 17: Método Elbow para películas.

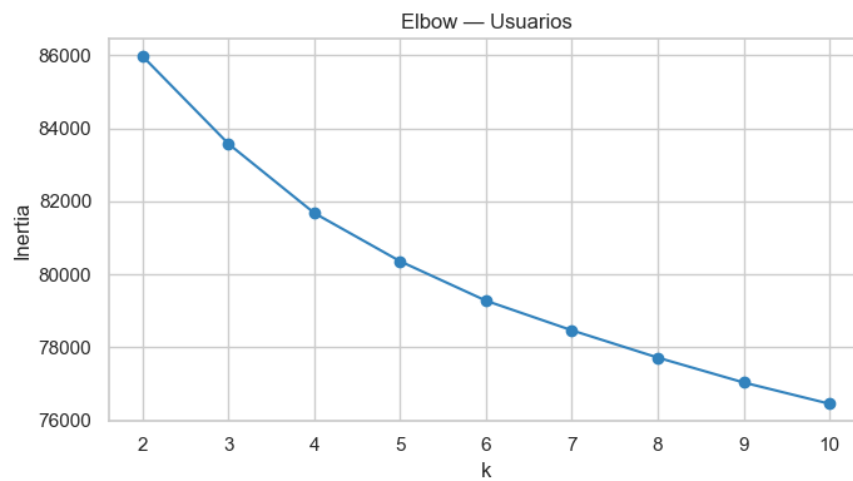


Figura 18: Método Elbow para usuarios.

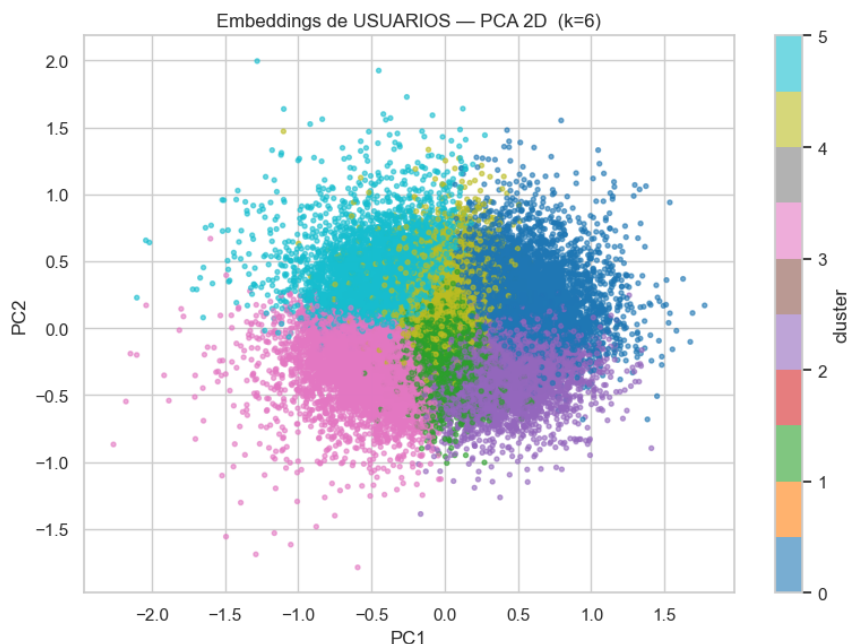


Figura 19: Embeddings de usuarios proyectados a 2D mediante PCA.

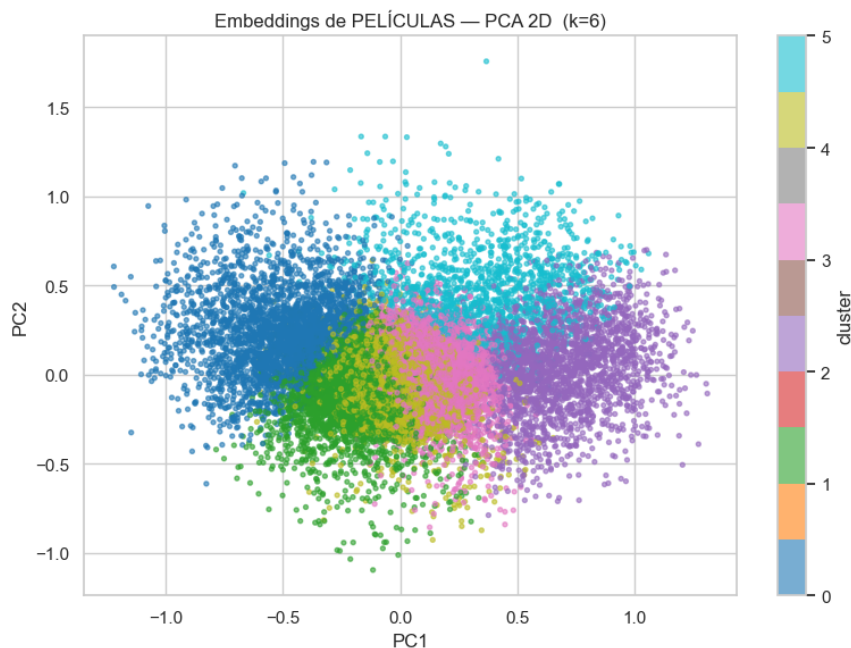


Figura 20: Embeddings de películas proyectados a 2D mediante PCA.

Los clústeres no son perfectamente separados, pero sí permiten construir comunidades interpretables para la interfaz: usuarios con gustos orientados a blockbusters, perfiles cinéfilos, nichos de género y películas de cola larga.

13.8. Recuperación Semántica y RAG

La evaluación de RAG se centra en la calidad de recuperación y cobertura. El pipeline reportó Precision@k semántico de 0,8833 y cobertura del Tag Genome de 0,25 en la corrida documentada. Esta cobertura limitada no invalida el componente: indica que la calidad depende de la disponibilidad de descriptores suficientes por película y debe monitorearse.

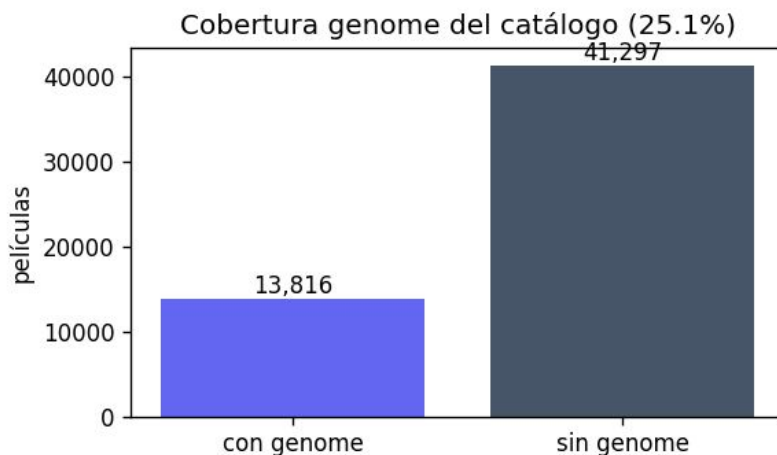


Figura 21: Cobertura del Tag Genome en el catálogo preparado.

La figura evidencia que solo 13.816 películas del catálogo preparado cuentan con información del Tag Genome, frente a 41.297 sin cobertura. Esta proporción aproximada de 25,1 % explica por qué el componente RAG debe monitorear cobertura y mantener fallbacks: cuando una película no tiene descriptores de genome suficientes, la recuperación semántica depende más de título, géneros y popularidad.

13.8.1. Ejemplos concretos de consultas RAG

Para validar que la recuperación semántica no depende únicamente de coincidencias literales de texto, se definieron consultas en lenguaje natural que combinan género, tono, intención y contexto emocional. En cada caso, el flujo esperado es: codificar la consulta con MiniLM, recuperar candidatos con FAISS, reordenar con prior de popularidad y, opcionalmente, generar una respuesta breve con RAG.

Consulta del usuario	Intención semántica	Respuesta esperada del sistema
“Películas de ciencia ficción oscuras y psicológicas”	Buscar ciencia ficción con tono serio, suspenso o conflicto mental.	Recuperar películas con tags como <i>sci-fi</i> , <i>psychology</i> , <i>dark</i> , <i>thought-provoking</i> ; explicar por qué encajan.
“Comedias románticas ligeras para ver en familia”	Combinar romance, comedia y baja carga dramática.	Priorizar películas accesibles, con géneros Comedy/Romance y tags positivos o familiares.
“Dramas bélicos bien valorados y emotivos”	Encontrar War/Drama con alto rating y carga emocional.	Mezclar similitud semántica con popularidad/rating para evitar recomendaciones irrelevantes de cola larga.
“Películas con final inesperado, misterio y tensión”	Buscar narrativa de suspenso, giros argumentales y misterio.	Recuperar películas asociadas a tags como <i>twist ending</i> , <i>suspense</i> , <i>mystery</i> y <i>thriller</i> .
“Animación de aventura con tono divertido”	Buscar películas animadas, familiares y dinámicas.	Recomendar películas de animación, aventura y público familiar con tags de humor y tono ligero.

Cuadro 6: Ejemplos de consultas usadas para validar la capa de búsqueda semántica y RAG.

Estos ejemplos cubren casos donde una búsqueda tradicional por título sería insuficiente. El valor del RAG no está solo en listar películas, sino en justificar la recomendación usando el contexto recuperado: géneros, tags relevantes y señales agregadas del catálogo.

13.9. Límites y Generalización

La matriz usuario–ítem tiene densidad muy baja, por lo que ningún modelo elimina completamente el cold-start. MovieLens representa usuarios activos de una plataforma de investigación y no necesariamente usuarios comerciales reales. Además, el dataset es estático; en producción se requerirían datos recientes, monitoreo continuo y retraining periódico.

13.10. Limitaciones éticas y sesgos

Un sistema de recomendación no es neutral: ordena la atención del usuario y puede influir en lo que una persona descubre, repite o deja de ver. Por esa razón, OmniRec debe evaluarse no solo por precisión, sino también por criterios éticos de diversidad, transparencia y responsabilidad.

- **Sesgo de popularidad:** los modelos colaborativos tienden a favorecer películas muy calificadas y conocidas. Esto puede invisibilizar obras de nicho, películas independientes o contenido con menor exposición histórica.
- **Desigualdad por género cinematográfico:** géneros con menor presencia o menor volumen de ratings, como documental, western, cine experimental o producciones menos comerciales, pueden recibir recomendaciones menos precisas por falta de evidencia histórica.
- **Sesgo cultural:** MovieLens refleja principalmente patrones de consumo de usuarios activos en un entorno dominado por cine estadounidense y europeo occidental. Esto puede subrepresentar cine latinoamericano, africano, asiático no masivo u otras cinematografías locales.
- **Burbujas de recomendación:** si el sistema insiste demasiado en gustos pasados, puede reducir la diversidad cultural y limitar el descubrimiento de géneros nuevos.
- **Representación incompleta del usuario:** MovieLens no contiene contexto personal, cultural o situacional. Inferir gustos profundos solo desde ratings puede simplificar demasiado a la persona usuaria.
- **Transparencia:** una recomendación debe poder explicarse de forma comprensible. Por eso se incorporan comunidades, tags, afinidad y búsqueda semántica como mecanismos de interpretación.
- **Riesgo de automatizar preferencias históricas:** si los datos reflejan hábitos pasados con sesgos, el modelo puede reproducirlos. El objetivo moral no debería ser encerrar al usuario en su historial, sino ayudarlo a descubrir contenido valioso.
- **Privacidad y minimización de datos:** aunque MovieLens es anónimo, una versión productiva debe recolectar solo lo necesario, evitar exposición de historiales sensibles y permitir que el usuario controle sus datos.
- **Responsabilidad sobre contenido recomendado:** el sistema no debe presentar sus sugerencias como verdades absolutas. La recomendación es una ayuda probabilística, no una decisión incuestionable.

Como mitigación ética, el sistema debe combinar precisión con diversidad, ofrecer explicaciones simples, mantener fallbacks transparentes, monitorear sesgos de popularidad y permitir que el usuario explore fuera de su perfil habitual. En términos morales, el recomendador debe ampliar opciones y autonomía, no manipular la atención ni reducir la experiencia cultural a maximización de clics.

13.11. Trabajo futuro

El sistema final cumple los objetivos del proyecto, pero deja líneas claras para una iteración posterior:

1. **Entrenar modelos profundos con mayor escala y GPU:** MF+Bias ya mostró ventaja en el protocolo validado; una evaluación futura podría entrenar arquitecturas neuronales con mayor volumen de datos y recursos dedicados para medir si la mejora se mantiene.
2. **Mejorar el cold-start con preferencias declaradas:** al registrar un usuario nuevo, se podrían solicitar géneros favoritos, películas conocidas o restricciones de contenido para inicializar recomendaciones antes de tener ratings históricos.
3. **Enriquecer la recuperación semántica en español:** añadir sinopsis, descripciones multilingües y metadatos externos ayudaría especialmente a consultas en español y a cine latinoamericano o menos representado en MovieLens.
4. **Incorporar controles de privacidad y eliminación de datos:** una versión productiva debería permitir que el usuario revise, exporte o elimine sus preferencias, respetando principios de minimización y control sobre datos personales.
5. **Evaluar diversidad y justicia además de precisión:** futuras métricas deberían medir cobertura de géneros, exposición de cola larga y diversidad intra-lista para evitar que el sistema optimice solo RMSE o popularidad.

14. Deployment

La fase de despliegue marca la transición del análisis en notebooks hacia una plataforma operativa. Respecto al informe anterior, se reemplaza la descripción inicial basada en Django/React/Inertia por la arquitectura final documentada: FastAPI, Next.js 16, Docker Compose, MLflow, registry y frontend OmniCine.

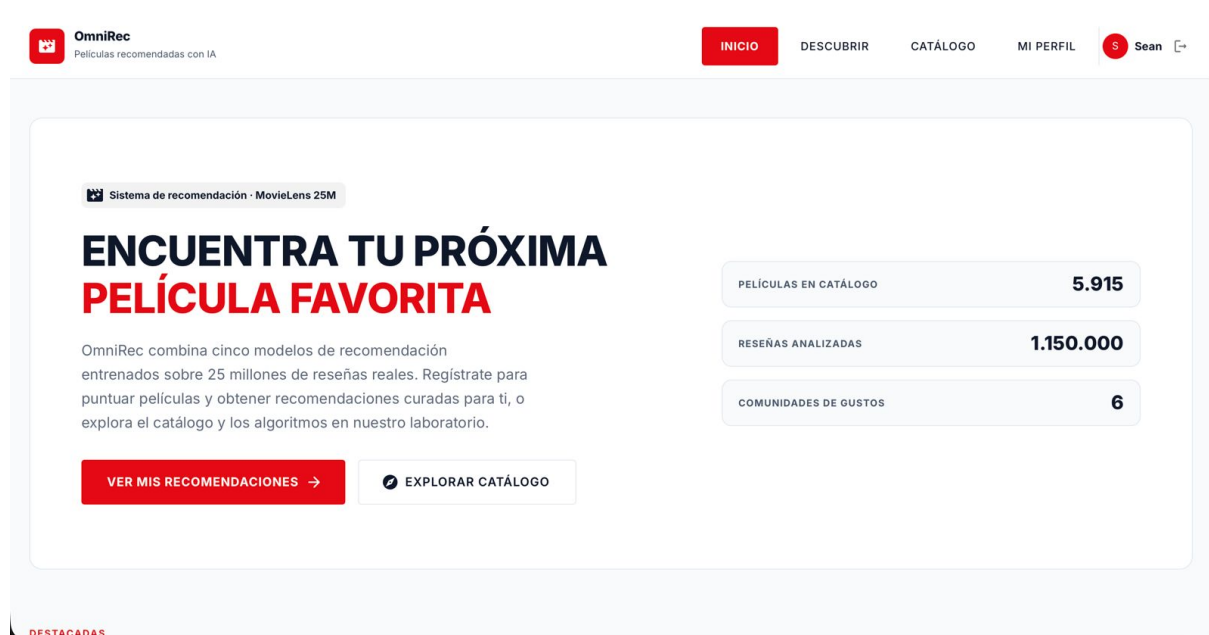


Figura 22: Pantalla inicial de la plataforma OmniRec / OmniCine.

14.1. Arquitectura del Sistema

- **Datos:** Parquets preparados a partir de MovieLens 25M.
- **Entrenamiento:** notebooks 03, 04 y 05, más pipeline en `src/`.
- **Artefactos:** modelos SVD, baseline, embeddings DL e índice RAG versionados en `models/registry/`.

- **Backend:** FastAPI con módulos de recomendación y recuperación semántica.
- **Frontend:** Next.js 16, interfaz OmniCine.
- **MLOps:** MLflow, configuración YAML, Docker, monitoreo y registry con hashes.

14.2. Flujos de Inferencia

14.2.1. 1. Motor de Recomendaciones

El endpoint de recomendaciones selecciona estrategia según el historial disponible. Si el usuario no tiene historial, se activa popularidad bayesiana. Si tiene ratings, SVD calcula afinidad personalizada con folding-in y filtra películas ya vistas.

14.2.2. 2. Perfilado de Comunidad

El sistema calcula un vector de usuario a partir de películas calificadas positivamente y lo compara contra centroides KMeans. La salida es una comunidad o arquetipo de gustos, usada en la sección de perfil.

14.2.3. 3. Similitud de Películas

La ficha de película usa embeddings MF+Bias para recuperar películas similares. Si el artefacto profundo no está disponible, el backend cae a similitud en el espacio latente de SVD.

14.2.4. 4. Búsqueda Semántica y RAG

La búsqueda convierte la consulta del usuario en embedding, consulta el índice FAISS y reordena resultados con un prior de popularidad. Si `generate=true` y existe API key, la capa generativa redacta una respuesta; si no, el sistema devuelve resultados estructurados sin fallar.

14.3. Ciclo de Vida del Usuario

1. El usuario entra como invitado o inicia sesión.
2. Explora catálogo y califica películas.
3. El backend sincroniza ratings y actualiza el perfil.
4. La sección Para Ti usa SVD o fallback de popularidad.
5. La ficha de película muestra similitud profunda.
6. La búsqueda semántica permite consultas naturales como “películas de ciencia ficción oscuras y psicológicas”.
7. La página de perfil muestra comunidad de gustos.

14.4. MLOps: Pipeline, Tracking y Registry

La capa MLOps hace que el sistema sea reproducible y auditable.

```
python -m src.pipeline all
make pipeline
make up-all
make jupyter
```

Las etapas principales son:

```
prepare -> train -> train_dl -> baseline -> index -> evaluate -> register -> monitor
```

MLflow registra métricas y parámetros de entrenamiento. El registry guarda cada artefacto con versión, hash SHA256, dataset, métricas y alias `current`. Los artefactos versionados incluyen:

Artefacto	Estado	Uso
<code>svd_model</code>	Current	Recomendaciones personalizadas y afinidad.
<code>baseline_scores</code>	Current	Cold-start y ranking de invitados.
<code>dl_embeddings</code>	Current	Similitud profunda entre películas.
<code>rag_index</code>	Current	Recuperación semántica.

Cuadro 7: Artefactos servidos y versionados.

14.5. Monitoreo y Política de Reentrenamiento

Señal monitoreada	Umbral
Caída de NDCG@10 contra versión <code>current</code>	> 5 %
Subida de RMSE del modelo DL contra <code>current</code>	> 5 %
Drift de distribución de ratings mediante PSI	> 0,2
Cobertura del recomendador o genome	< 0,6
Frescura de datos desde el último rating	> 30 días

Cuadro 8: Umbrales de monitoreo del sistema.

La política de reentrenamiento considera nuevos ratings acumulados, calendario mensual de respaldo y guardrails de calidad: una nueva versión solo se promueve a `current` si no degrada métricas críticas.

14.6. Especificaciones Técnicas de Despliegue

Componente	Tecnología
Modelado clásico	scikit-surprise, Pandas, Parquet
Deep Learning	PyTorch 2.4, embeddings MF+Bias y NeuMF
Recuperación	Sentence-Transformers MiniLM, FAISS, RAG generativo opcional
Backend	FastAPI, módulos <code>engine</code> y <code>semantic</code>
Frontend	Next.js 16, OmniCine
MLOps	MLflow, registry SHA256, YAML, Docker Compose
Persistencia operativa	Parquet, modelos serializados, registry local

Cuadro 9: Stack técnico final.

15. Conclusiones

El proyecto final cumple el alcance planteado al inicio: integra Machine Learning clásico, Deep Learning, AutoML, MLOps, recuperación semántica, RAG y despliegue web. La evolución más importante respecto al informe anterior es que MLOps ya no aparece como una propuesta, sino como un ciclo operativo con pipeline, MLflow, versionado de artefactos, Docker y monitoreo.

SVD se mantiene como el modelo clásico principal para recomendación personalizada. MF+Bias demuestra que Deep Learning aporta valor cuando se compara bajo el mismo protocolo. La capa MiniLM + FAISS + RAG amplía el sistema hacia búsqueda por significado, resolviendo consultas que no pueden representarse únicamente con ratings. La web OmniCine integra estos modelos en flujos concretos: recomendaciones para usuarios, fallback para invitados, comunidades, similitud de películas y búsqueda semántica.

Las principales limitaciones siguen siendo la dispersión, el cold-start, la cobertura incompleta del Tag Genome y el sesgo de popularidad. Sin embargo, el diseño final mitiga estos riesgos con modelos especializados, fallbacks defensivos y monitoreo continuo.

Referencias

- [Barocas et al., 2023] Barocas, S., Hardt, M., and Narayanan, A. (2023). *Fairness and Machine Learning: Limitations and Opportunities*. MIT Press.
- [Harper and Konstan, 2015] Harper, F. M. and Konstan, J. A. (2015). The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4):19:1–19:19.
- [He et al., 2017] He, X., Liao, L., Zhang, H., Nie, L., Hu, X., and Chua, T.-S. (2017). Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web*, pages 173–182.
- [Hug, 2020] Hug, N. (2020). Surprise: A python library for recommender systems. *Journal of Open Source Software*, 5(52):2174.
- [Johnson et al., 2021] Johnson, J., Douze, M., and Jégou, H. (2021). Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547.
- [Koren et al., 2009] Koren, Y., Bell, R., and Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37.
- [Reimers and Gurevych, 2019] Reimers, N. and Gurevych, I. (2019). Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992.
- [Sculley et al., 2015] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, volume 28, pages 2503–2511.